Survey Report: Grandmaster Level in StarCraft II Using Multi-Agent Reinforcement Learning

A Comprehensive Summary of the AlphaStar System

ECE 752 — Advanced Computer Architecture

Summary of Vinyals, O. et al. (Nature, 2019)

Abstract

This report provides a comprehensive survey of the DeepMind AlphaStar system, as presented in "Grandmaster Level in StarCraft II Using Multi-Agent Reinforcement Learning" (Vinyals et al., *Nature* 575, 350–354, 2019). AlphaStar is the first artificial intelligence agent to reach Grandmaster level in the full game of StarCraft II, placing above 99.8% of officially ranked human players across all three playable races. The system combines deep neural network architectures, imitation learning from human replays, multi-agent reinforcement learning via a novel league training procedure, and operates under human-like constraints including camera restrictions and action-rate limits. This report details the game environment, the agent architecture, the training pipeline, the multi-agent league framework, empirical evaluation results, ablation studies, and broader implications for AI research.

Contents

1 Introduction **2**

2 The StarCraft II Domain **2**
2.1 Game Overview . . . 2
2.2 Challenges for AI . . . 2
2.3 Observation and Action Spaces . . . 3

3 AlphaStar Architecture **3**
3.1 Input Encoders . . . 4
3.2 Core: Deep LSTM . . . 4
3.3 Output Heads (Auto-Regressive Decoding) . . . 4
3.4 Value Network . . . 5

4 Training Pipeline **5**
4.1 Phase 1: Supervised Learning . . . 5
4.2 Phase 2: Reinforcement Learning . . . 6
4.3 Phase 3: League Training (Multi-Agent RL) . . . 6
4.3.1 Prioritized Fictitious Self-Play (PFSP) . . . 6
4.3.2 Agent Types . . . 7

5 Human-Like Interface Constraints **7**

6 Empirical Results **8**
6.1 Battle.net Evaluation . . . 8
6.2 Performance Progression . . . 8
6.3 Actions Per Minute (APM) Comparison . . . 8

7 Ablation Studies **9**
7.1 Key Ablation Findings 9

8 Training Infrastructure **10**

9 Training Progression and Strategic Diversity **11**
9.1 Elo Progression During League Training 11
9.2 Robustness to Unseen Strategies 11
9.3 Nash Distribution and Strategic Diversity 11

10 Comparison with Related Work **11**

11 Discussion and Broader Implications **12**
11.1 Generality of the Approach 12
11.2 Limitations 12
11.3 Professional Player Assessment 12

12 Conclusion **12**

1 Introduction

Many real-world applications require artificial agents to compete and coordinate with other agents in complex, partially observable environments. StarCraft II, developed by Blizzard Entertainment, has long served as a grand challenge for artificial intelligence research. It is one of the most popular and difficult professional esports, featuring:

- A **combinatorial action space** with approximately 10^{26} possible choices at each step.
- **Imperfect information** due to fog of war and a camera-based interface.
- **Long time horizons** spanning tens of thousands of time-steps per game ($\sim$10 minutes of real-time play).
- **Non-transitive strategy dynamics** (rock–paper–scissors cycles among strategies).
- **Three distinct races** (Terran, Protoss, Zerg) with fundamentally different mechanics.

Prior to AlphaStar, the strongest AI agents for StarCraft either simplified important aspects of the game, used superhuman capabilities (e.g., viewing the entire map simultaneously or issuing thousands of actions per minute), or relied on hand-crafted sub-systems. No previous agent came close to matching the overall skill of top human players in the unmodified game.

AlphaStar addresses these challenges using *general-purpose* machine learning methods—combining neural network architectures, imitation learning, reinforcement learning, and multi-agent training—that are in principle applicable to any complex domain. The resulting agent was evaluated on Blizzard's official Battle.net matchmaking system and achieved Grandmaster-level ratings for all three races, placing above 99.8% of ranked human players on the European server.

2 The StarCraft II Domain

2.1 Game Overview

StarCraft II is a real-time strategy (RTS) game in which players balance high-level economic decisions with individual control of hundreds of units. The most common competitive setting is 1v1, where each player selects one of three races:

- **Terran**: Human-like faction with flexible, mechanized units.
- **Protoss**: Technologically advanced race with powerful but expensive units.
- **Zerg**: Biological swarm race emphasizing rapid production and adaptation.

Players begin with a small base and a few worker units, gather resources, construct buildings, research technologies, scout opponents, and build armies. A player is defeated when all their buildings are destroyed.

2.2 Challenges for AI

Figure 1 illustrates the key challenges that make StarCraft II a formidable AI benchmark.

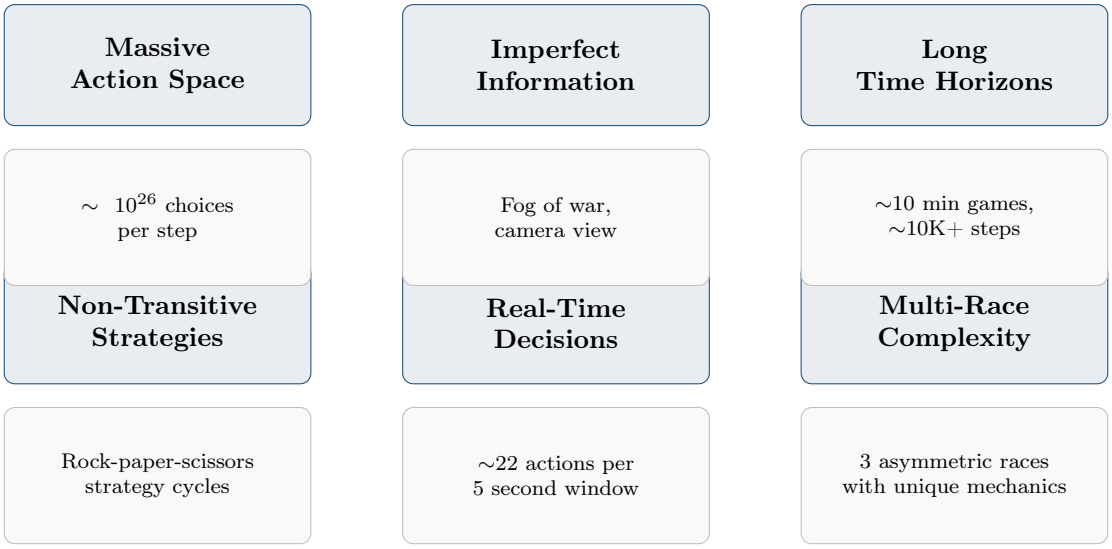

Figure 1: Key challenges of StarCraft II as an AI benchmark. The combination of these factors makes it substantially harder than previous game-playing benchmarks such as chess, Go, or Atari.

2.3 Observation and Action Spaces

At each time step t, the agent receives an observation o_t that includes:

- A list of all observable units and their attributes (up to 512 entities).
- A 128×128 spatial map with height, visibility, creep, entity ownership, and pathability layers.
- Player data: race, upgrades, resource counts, supply, and army statistics.
- Game statistics: camera position and in-game time.

Each action a_t is highly structured, consisting of:

- **Action type**: what to do (move, attack, build, etc.).
- **Selected units**: which units execute the action.
- **Target**: an entity or map location (discretized to 256×256).
- **Queued**: whether to queue or execute immediately.
- **Delay**: how many time-steps to wait before the next observation.

This representation results in approximately 10^{26} possible choices at each step, making direct enumeration infeasible.

3 AlphaStar Architecture

Central to AlphaStar is a policy $\pi_\theta(a_t \mid s_t, z) = \mathbb{P}[a_t \mid s_t, z]$, implemented as a deep neural network with parameters θ. The network receives all observations from the start of the game as inputs and selects actions as outputs. The policy is also conditioned on a statistic z that summarizes a strategy sampled from human data (e.g., a build order). Figure 2 provides a high-level overview of the architecture.

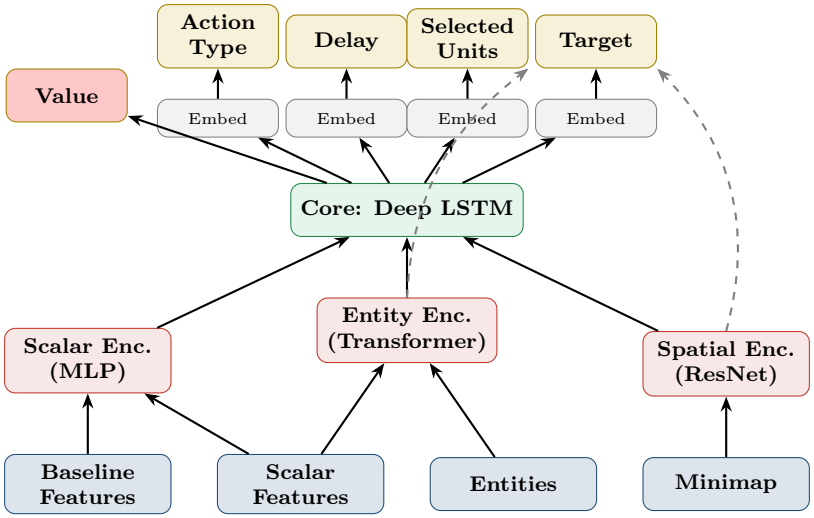

Figure 2: High-level overview of the AlphaStar neural network architecture. Observations are processed by specialized encoders (scalar MLP, entity Transformer, spatial ResNet), integrated through a deep LSTM core, and decoded into structured action outputs via auto-regressive heads. Skip connections (dashed) allow direct information flow from entity encoders to the pointer network (selected units) and from spatial encoders to the target head. The model has 139 million parameters total, though only 55 million are needed during inference.

3.1 Input Encoders

- **Scalar Encoder (MLP)**: Processes non-spatial features such as resource counts, supply, race information, and the strategy statistic z.
- **Entity Encoder (Transformer)**: Processes the variable-length list of up to 512 game entities using self-attention mechanisms, with scatter connections to integrate spatial and non-spatial information.
- **Spatial Encoder (ResNet)**: Processes the 128×128 minimap layers using a residual convolutional network.

3.2 Core: Deep LSTM

The encoded representations are combined and fed into a deep Long Short-Term Memory (LSTM) network. The LSTM maintains memory across time steps, which is critical for handling partial observability—the agent must remember information about unseen areas of the map and track opponent activities over time.

3.3 Output Heads (Auto-Regressive Decoding)

The action space is decomposed into structured sub-actions, each predicted by a specialized output head conditioned on previous sub-action selections:

1. **Action Type** (Residual MLP): Selects what action to perform.
2. **Delay** (MLP): Determines when to observe next.
3. **Queued** (MLP): Whether to queue the action.
4. **Selected Units** (Pointer Network): Which units to command.
5. **Target Unit** (Attention): Which entity to target.
6. **Target Point** (Deconvolutional ResNet): Where on the map to act.

This auto-regressive decomposition via the chain rule reduces the intractable 10^{26}-dimensional action space into a sequence of manageable categorical decisions.

3.4 Value Network

A separate value function $V_\theta(s_t, z)$ estimates the expected outcome of the game from the current state. During training, the value function uses *both* the player's and the opponent's observations to reduce variance, though opponent information is not used during evaluation.

4 Training Pipeline

AlphaStar's training proceeds in two main phases: supervised learning from human replays, followed by reinforcement learning within a multi-agent league. Figure 3 illustrates this pipeline.

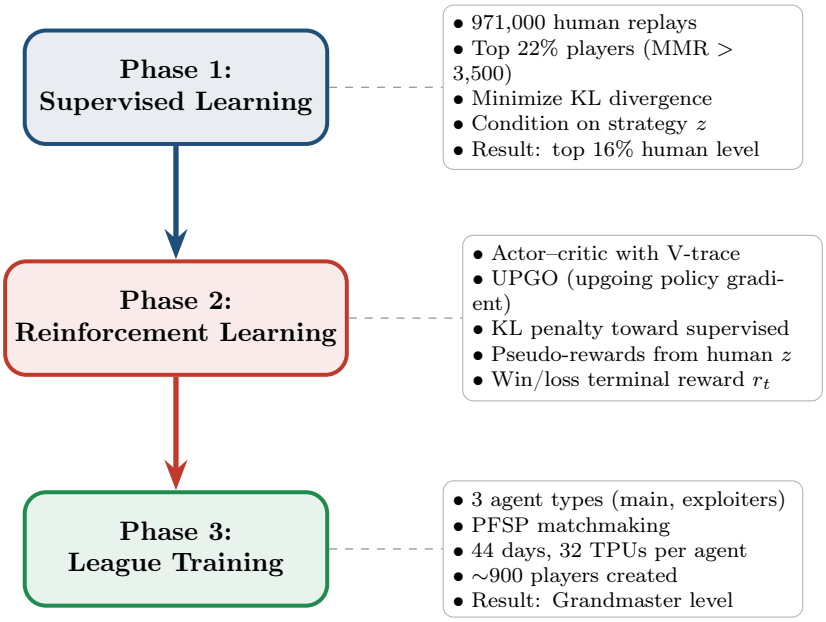

Figure 3: The three-phase training pipeline of AlphaStar. Supervised learning bootstraps the policy from human replays, reinforcement learning refines it through self-play, and league training produces a diverse, robust population of agents that avoid strategic cycling.

4.1 Phase 1: Supervised Learning

AlphaStar is initially trained on a publicly available dataset of 971,000 replays from StarCraft II, filtered to games by players with MMR scores above 3,500 (top 22%). One policy is trained per race with the same architecture used during reinforcement learning. Key details:

- For each replay, a statistic z is extracted encoding the player's build order (first 20 constructed buildings and units) and cumulative statistics (units, buildings, effects, and upgrades present during the game).
- The policy is trained to predict each human action a_t conditioned on either s_t alone or on (s_t, z), with z set to zero 10% of the time.
- The loss is the KL divergence between the model's output distribution and the human actions, optimized with Adam and L_2 regularization.
- Fine-tuning on replays with MMR above 6,200 (16,000 games) further improves win rate from 87% to 96% against built-in Protoss vs. Protoss elite bots.

The supervised agents achieved fine-tuned MMR ratings of 3,947 (Terran), 3,607 (Protoss), and 3,544 (Zerg), placing them in approximately the top 16% of human players.

4.2 Phase 2: Reinforcement Learning

Reinforcement learning refines the supervised policy to maximize win rate. The terminal reward is $r_T \in \{-1, 0, +1\}$ (loss, draw, win) with no discount. Key algorithmic components:

- **Actor–Critic Framework**: Based on the advantage actor–critic algorithm, with updates applied asynchronously on replayed experience (off-policy learning).
- **V-trace**: Clipped importance sampling to correct for the mismatch between the behavior policy and the current policy in the large action space.
- **TD(λ)**: Temporal difference learning for value function estimation.
- **UPGO (Upgoing Policy Gradient)**: A novel algorithm that updates the policy only from partial trajectories with better-than-expected returns, bootstrapping when the behavior policy takes a worse-than-average action. This helps the agent learn from successful sub-trajectories.
- **KL Regularization**: A penalty is applied whenever the agent's action probabilities differ from the supervised policy, maintaining exploration of human-like strategies throughout training.
- **Pseudo-Rewards**: Agents receive additional rewards for following the strategy statistic z (measured by edit distance between sampled and executed build orders, and Hamming distance on cumulative statistics), encouraging diverse strategic exploration.

The overall loss is a weighted sum of the win-loss reward, the V-trace policy and value losses, UPGO, pseudo-reward losses, the KL divergence with the supervised policy, and entropy regularization.

4.3 Phase 3: League Training (Multi-Agent RL)

League training is a novel multi-agent reinforcement learning framework designed to address the non-transitive strategy dynamics in StarCraft II (where strategy A beats B, B beats C, but C beats A). Figure 4 illustrates the league structure.

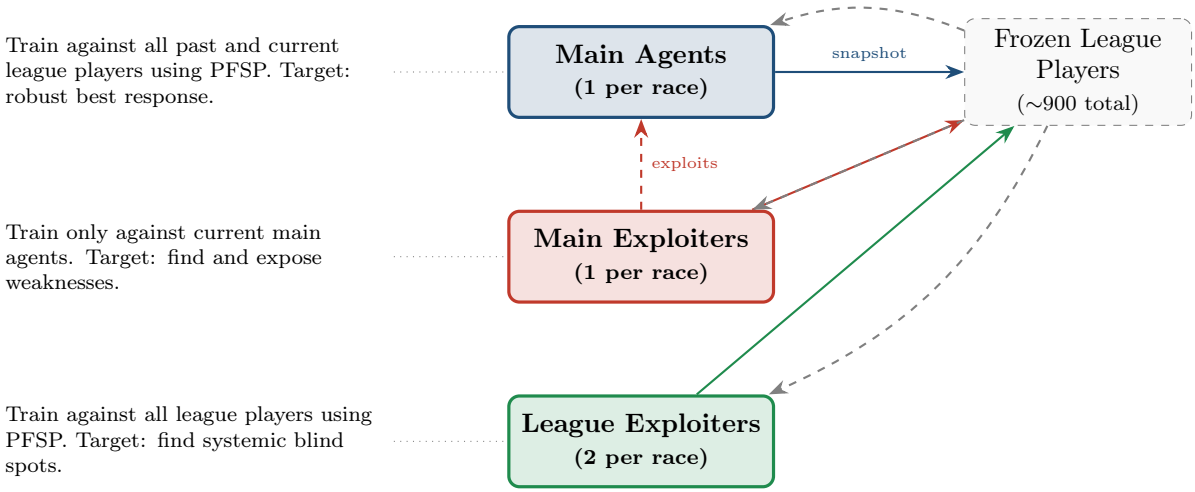

Figure 4: Structure of the AlphaStar league. Three types of agents interact: main agents seek robust performance against the full league, main exploiters specifically target weaknesses in main agents, and league exploiters search for systemic blind spots across the entire population. All agents periodically snapshot frozen copies into the league, growing the pool of diverse opponents.

4.3.1 Prioritized Fictitious Self-Play (PFSP)

Standard Fictitious Self-Play (FSP) computes a best response against a uniform mixture of all previous policies, which converges to a Nash equilibrium in two-player zero-sum games.

AlphaStar extends this with **Prioritized FSP (PFSP)**, which adapts the opponent mixture proportionally to the win rate of each opponent against the agent:

$$f_{\text{hard}}(x) = (1 - x)^p, \quad p \in \mathbb{R}_+ \tag{1}$$

where x is the agent's current win rate against the opponent. This focuses training on the hardest opponents—those the agent currently struggles against—rather than wasting games against already-defeated opponents. This is critical in StarCraft because the pursuit of maximum *average* win rate leads to easily exploitable policies in a game with highly non-transitive dynamics.

4.3.2 Agent Types

The league consists of three agent types with different roles:

1. **Main Agents**: Use PFSP (35% self-play, 50% PFSP, 15% forgotten-player PFSP) against all players. These agents are never reset and form the final deployed policy.
2. **Main Exploiters**: Train only against current main agents to identify potential weaknesses. They are periodically reset to supervised parameters, making main agents more robust.
3. **League Exploiters**: Use PFSP against the entire league to find global blind spots. They too are periodically reset, helping discover specialist strategies that are not necessarily robust themselves.

During 44 days of league training using 12 separate actor–learner setups (each with 32 TPUs), approximately 900 distinct players were created in the league.

5 Human-Like Interface Constraints

A critical design decision in AlphaStar is to impose constraints that approximate human physical limitations, ensuring fair comparison. Figure 5 summarizes these constraints.

Human-Like Constraints Applied to AlphaStar

Camera View: Agent uses a camera-like interface, seeing only a 32×20 game-unit region at a time.

APM Limit: Maximum ∼22 non-duplicate actions per 5-second window (∼264 APM peak).

Observation Delay: ∼110 ms delay between observation and action execution (network + processing).

Lookahead Delay: Agent decides ∼370 ms in advance when to next observe (average).

Imperfect Information: Same fog-of-war rules as humans—no hidden unit information visible.

Figure 5: Summary of the constraints imposed on AlphaStar to ensure a fair comparison with human players. These constraints were developed in consultation with professional StarCraft II players and Blizzard employees.

Ablation studies showed that these constraints significantly affect performance:

- Removing the camera interface increased win rate from 87% to 94% against the elite bot, demonstrating that camera management is a substantial challenge.

- Reducing APM substantially reduced performance, and unexpectedly, *increasing* APM beyond the limit also reduced performance—because the agent spent more effort on micro-tactics rather than diverse strategic play.

6 Empirical Results

6.1 Battle.net Evaluation

AlphaStar was evaluated at three training snapshots on Blizzard's official Battle.net matchmaking system under blind conditions (anonymous account, no opponent identity information):

Table 1: AlphaStar MMR ratings and human population percentiles on Battle.net. Grandmaster (GM) threshold is approximately 6,200 MMR on the European server.

Agent	**Training**	**Protoss**	**Terran**	**Zerg**	**Percentile**
AlphaStar Supervised	SL only	3,607	3,947	3,544	∼84%
AlphaStar Mid	SL + 27d RL	5,991	6,048	5,755	∼99.5%
AlphaStar Final	SL + 44d RL	**6,275**	**6,048**	**5,835**	**> 99.8%**
Grandmaster Threshold	—		∼6,200		99.8%

AlphaStar Final achieved Grandmaster level for all three races, with its highest rating of 6,275 MMR for Protoss. This places it above 99.8% of approximately 90,000 ranked players on the European server.

6.2 Performance Progression

Figure 6 shows how AlphaStar's performance improved over the course of training.

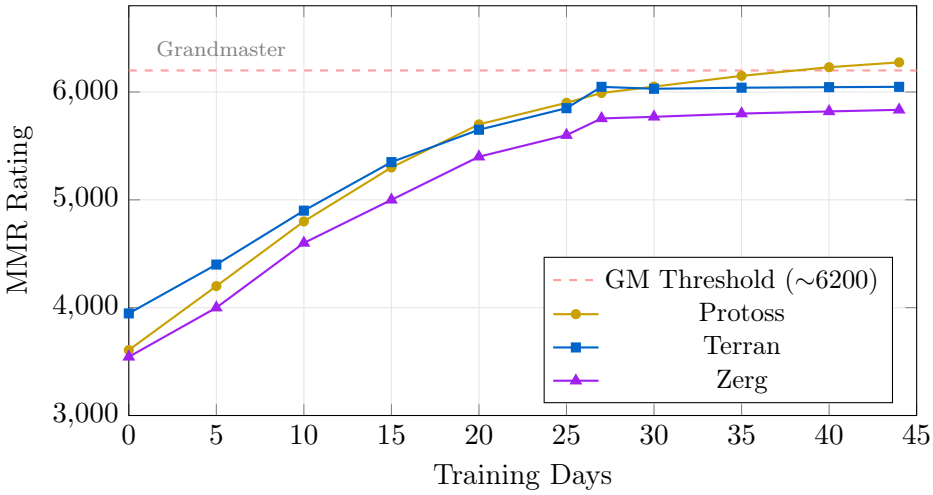

Figure 6: Approximate MMR progression of AlphaStar over 44 days of league training for each race. Day 0 corresponds to the supervised learning baseline. All three races surpass or approach the Grandmaster threshold (∼6,200 MMR) by the end of training.

6.3 Actions Per Minute (APM) Comparison

AlphaStar's effective APM was constrained to be comparable to or lower than human professionals. As reported by StarCraft II:

Table 2: Effective actions per minute (EPM) statistics for AlphaStar Final (blue) and human opponents (red) on Battle.net.

	Average EPM	**99.9th %ile EPM**	**Max EPM**
Terran — AlphaStar	183	487	571
Terran — Human	200	554	755
Protoss — AlphaStar	187	509	587
Protoss — Human	205	537	621
Zerg — AlphaStar	211	655	823
Zerg — Human	259	823	1192

Notably, AlphaStar's peak APM statistics are *substantially lower* than those of human opponents, confirming that the agent does not rely on superhuman speed.

7 Ablation Studies

Extensive ablation experiments were conducted on a simplified setup (one map, Protoss vs. Protoss, 10^{10} training steps, with Elo ratings estimated against 11,000 games with the built-in StarCraft II bots). Figure 7 summarizes the key findings.

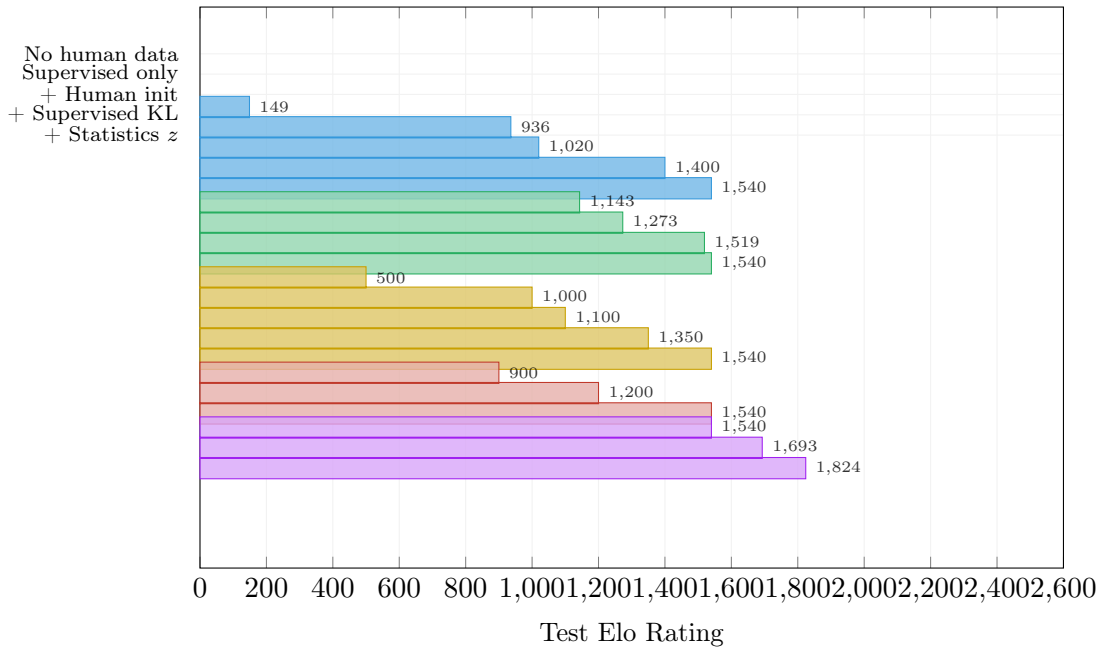

Figure 7: Summary of ablation study results (Test Elo). Each bar shows the performance contribution of a component. Human data, PFSP-based multi-agent learning, Transformer-based entity encoding with scatter connections, UPGO, and exploiter agents all provide significant improvements.

7.1 Key Ablation Findings

1. **Human Data is Critical**: Removing all human data collapses performance to Elo 149. Even the supervised-only agent (Elo 936) benefits enormously from human initialization (1,020), KL regularization (1,400), and strategy conditioning z (1,540).

2. **Multi-Agent Learning**: PFSP + self-play (Elo 1,540) substantially outperforms pure self-play (1,519) and especially pure FSP (1,143). Adding exploiters raises league Elo from 1,540 (main agents only) to 1,824 (full league with all exploiters), an 18% improvement.
3. **Architecture**: The Transformer-based entity encoder, pointer network for unit selection, and scatter connections each contribute significantly. The baseline architecture without these components achieves only Elo ∼500.
4. **Off-Policy Learning**: V-trace alone provides reasonable off-policy correction, but adding TD(λ) and UPGO together yield the best combined performance.
5. **Value Function**: Using opponent information as input to the value function (during training only) improves average win rate from 25% to 82%, a dramatic reduction in value estimation variance.
6. **Camera Interface**: Using the non-camera interface yields a 94% win rate vs. the elite bot, compared to 87% with the camera—confirming that camera management is a meaningful component of skill.
7. **APM Limits**: Reducing APM reduces performance as expected, but *increasing* APM also reduces performance, as the agent spends more effort on micro-tactics rather than strategic diversity.

8 Training Infrastructure

The scale of computation required to train AlphaStar is substantial. Figure 8 illustrates the distributed training infrastructure.

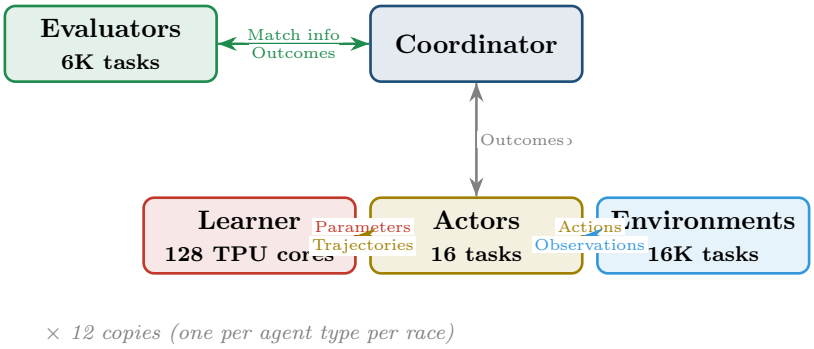

Figure 8: Distributed training infrastructure for AlphaStar's league. Each of the 12 agent instances uses a 128-core TPU learner, 16 actor tasks, and access to 16K environment tasks. A central coordinator manages matchmaking and tracks payoff matrices. Evaluators (6K tasks) run on CPUs to estimate Elo ratings.

Key infrastructure details:

- **Per agent**: 16,000 concurrent StarCraft II matches, 16 actor tasks (each using a TPU v3 with 8 cores), and a 128-core TPU learner.
- **Total**: 12 actor–learner setups (1 main agent + 1 main exploiter + 2 league exploiters per race, × 3 races).
- **Throughput**: Each TPU core processes a mini-batch of 4 sequences (batch size 512 total). Actors update parameters every 10 seconds.
- **Game instances**: Run asynchronously on preemptible CPUs (∼150 processors, ≡ 28 physical cores each).
- **Duration**: 44 days of league training.

9 Training Progression and Strategic Diversity

9.1 Elo Progression During League Training

Over the 44 days of league training, the Elo scores of agents in the league steadily increased. Main agents showed consistent improvement, eventually reaching Elo scores above 1,600 (relative to the elite built-in bot at Elo 0). League exploiters and main exploiters showed more variable trajectories, as they are periodically reinitialized.

9.2 Robustness to Unseen Strategies

The proportion of validation strategies (17 hand-curated strategies including cannon rush and early flying units) that the main agents could defeat in more than 80 out of 160 games increased steadily over training time. By the end:

- **Protoss**: ∼100% of validation strategies beaten.
- **Terran**: ∼95% of validation strategies beaten.
- **Zerg**: ∼90% of validation strategies beaten.

9.3 Nash Distribution and Strategic Diversity

The Nash distribution (the mixture of least-exploitable players) shifted dynamically during training, with the latest strategies largely dominating earlier ones. The unit composition also changed throughout training—unlike main agents which converged on common unit types, exploiters rapidly explored diverse and unusual unit compositions, driving strategic innovation in the league.

10 Comparison with Related Work

Table 3: Comparison of AlphaStar with other game-playing AI systems.

System	Game	Approach	Simplifications	Level Achieved
Deep Blue	Chess	Search + evaluation	None (perfect info)	World champion
AlphaGo/Zero	Go	MCTS + RL	None (perfect info)	Superhuman
OpenAI Five	Dota 2	RL (PPO)	Restricted heroes, simplified rules	99.4% of players
AlphaStar	**SC2**	**IL + RL + League**	**None (full game)**	**Grandmaster (99.8%)**

Key distinctions of AlphaStar:

- **No game simplifications**: Unlike OpenAI Five (restricted hero pool, no items, simplified rules), AlphaStar plays the full, unmodified game.
- **No superhuman capabilities**: Human-like camera, APM limits, and observation delays.
- **Model-free, end-to-end**: No search, no hand-crafted components, no rule-based subsystems.
- **All three races**: Separate agents for Terran, Protoss, and Zerg, all achieving Grandmaster.

11 Discussion and Broader Implications

11.1 Generality of the Approach

The techniques used in AlphaStar are designed to be general-purpose and applicable beyond StarCraft II:

- **Neural architecture**: The Transformer-based entity encoder with scatter connections applies to any domain with variable-length sets of objects (images, lists, sets).
- **Auto-regressive action decomposition**: Any domain with structured, combinatorial action spaces can use chain-rule decomposition with pointer networks.
- **League training with PFSP**: Applicable to any multiplayer domain with non-transitive dynamics.
- **Imitation + RL**: The combination of supervised pre-training with reinforcement learning fine-tuning is widely applicable wherever human demonstrations are available.
- **Off-policy components**: V-trace, UPGO, and distillation toward a supervised policy are general RL techniques.

11.2 Limitations

- **Computational cost**: Training required 12×32 TPUs for 44 days—enormous by academic standards.
- **Human data dependency**: Removing human data collapsed performance, suggesting pure RL exploration is insufficient for this domain.
- **Camera interface limitations**: While AlphaStar uses a camera, it can target locations outside the camera more accurately than humans (selected on a 256×256 grid), and treats all units identically regardless of camera position.
- **Single-agent deployment**: Only the unconditional main agent (not the full league) is deployed for evaluation.

11.3 Professional Player Assessment

Dario "TLO" Wünsch, a professional StarCraft II player and co-author of the paper, stated:

> *"While AlphaStar has excellent and precise control it doesn't feel superhuman—certainly not on a level that a human couldn't theoretically achieve. It is better in some aspects than humans and then also worse in others [...] Overall, it feels very fair, like it is playing a 'real' game of StarCraft and doesn't completely throw the balance off by having unrealistic capabilities."*

12 Conclusion

AlphaStar represents a landmark achievement in artificial intelligence: the first agent to reach Grandmaster level in the full, unmodified game of StarCraft II, and the first to reach the highest league of human players in a major professional esport without simplification. The system demonstrates that general-purpose machine learning techniques—combining deep neural network architectures, imitation learning from human demonstrations, multi-agent reinforcement learning with league training and PFSP, and careful algorithmic innovations like UPGO—can tackle domains that require real-time decisions, imperfect information, combinatorial action spaces, and complex multi-agent strategy dynamics.

The success of AlphaStar suggests that similar approaches may have substantial impact on real-world problems sharing these characteristics, such as autonomous driving, robotic control, personal assistants, and other domains requiring real-time decision-making under uncertainty with structured, combinatorial actions.

References

[1] Vinyals, O. et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature* **575**, 350–354 (2019).

[2] Silver, D. et al. Mastering the game of Go with deep neural networks and tree search. *Nature* **529**, 484–489 (2016).

[3] Mnih, V. et al. Human-level control through deep reinforcement learning. *Nature* **518**, 529–533 (2015).

[4] Vinyals, O. et al. StarCraft II: a new challenge for reinforcement learning. Preprint at `https://arxiv.org/abs/1708.04782` (2017).

[5] Sutton, R. & Barto, A. *Reinforcement Learning: An Introduction* (MIT Press, 1998).

[6] LeCun, Y., Bengio, Y. & Hinton, G. Deep learning. *Nature* **521**, 436–444 (2015).

[7] Vaswani, A. et al. Attention is all you need. *Adv. Neural Information Process. Syst.* **30**, 5998–6008 (2017).

[8] Espeholt, L. et al. IMPALA: scalable distributed deep-RL with importance weighted actor-learner architectures. *Proc. Machine Learning Res.* **80**, 1407–1416 (2018).

[9] OpenAI. OpenAI Five. `https://blog.openai.com/openai-five/` (2018).

[10] Jaderberg, M. et al. Human-level performance in 3D multiplayer games with population-based reinforcement learning. *Science* **364**, 859–865 (2019).